VoiceScribe AI

Piattaforma on-premise di trascrizione e diarizzazione audio/video

Architettura a Microservizi Riutilizzabili — OpenAPI/Swagger

Documento di Progettazione Tecnica — v2.0.0

Prodotto	**VoiceScribe AI**
Versione documento	2.0.0 — Microservizi + OpenAPI
Numero microservizi	**8 — tutti autonomi e riutilizzabili in altri progetti**
API Standard	**OpenAPI 3.1.0 — Swagger UI + ReDoc per ogni servizio**
Hardware Target	AMD Threadripper + NVIDIA RTX 6000 Ada (48 GB VRAM)
Stack Core	Python · FastAPI · Celery · Redis · PostgreSQL · FFmpeg · Pyannote
Modello di Accesso	Free (2/giorno, 30 min) · Pro · Enterprise

Documento riservato — uso interno

1. EXECUTIVE SUMMARY

VoiceScribe AI è una piattaforma di trascrizione audio/video on-premise ad altissime prestazioni, progettata per girare in modalità bare metal su Linux sfruttando un processore AMD Threadripper e una GPU NVIDIA RTX 6000 con 48 GB di VRAM. Il sistema è concepito per aziende che trattano grandi volumi di registrazioni — call center, riunioni, interviste, podcast, supporto tecnico — e necessitano di trascrizioni accurate, veloci e con identificazione degli speaker, senza dipendenze da servizi cloud.

La versione 2.0 introduce due pilastri architetturali fondamentali: l'organizzazione in 8 microservizi completamente indipendenti, ognuno con il proprio contratto OpenAPI 3.1.0, e il modello freemium con un Free Tier pubblico (2 trascrizioni/giorno fino a 30 minuti). Ogni microservizio è progettato per essere estratto e riutilizzato in altri progetti senza portare con sé dipendenze da VoiceScribe AI.

Caratteristiche Chiave v2.0

- 8 microservizi autonomi — ciascuno con openapi.yaml, Dockerfile, requirements.txt indipendenti
- OpenAPI 3.1.0 per ogni servizio — Swagger UI su /docs, ReDoc su /redoc, JSON su /openapi.json
- Riutilizzabilità by design — SVC-06 (Whisper) e SVC-07 (Pyannote) funzionano standalone in qualsiasi progetto
- Free Tier: 2 trascrizioni/giorno fino a 30 min — contatore Redis atomico, reset a mezzanotte UTC
- RTF ~0.023: file da 30 min trascritto in ~42 secondi su RTX 6000 con Whisper large-v3 int8_float16

2. ARCHITETTURA A MICROSERVIZI

VoiceScribe AI è composto da 8 microservizi indipendenti. Ogni servizio ha un perimetro di responsabilità netto, un'interfaccia API documentata con OpenAPI 3.1.0, e nessuna dipendenza diretta da altri servizi del sistema — comunica esclusivamente tramite chiamate REST o code Celery/Redis. Questa separazione consente di estrarre qualsiasi servizio e integrarlo in un progetto completamente diverso.

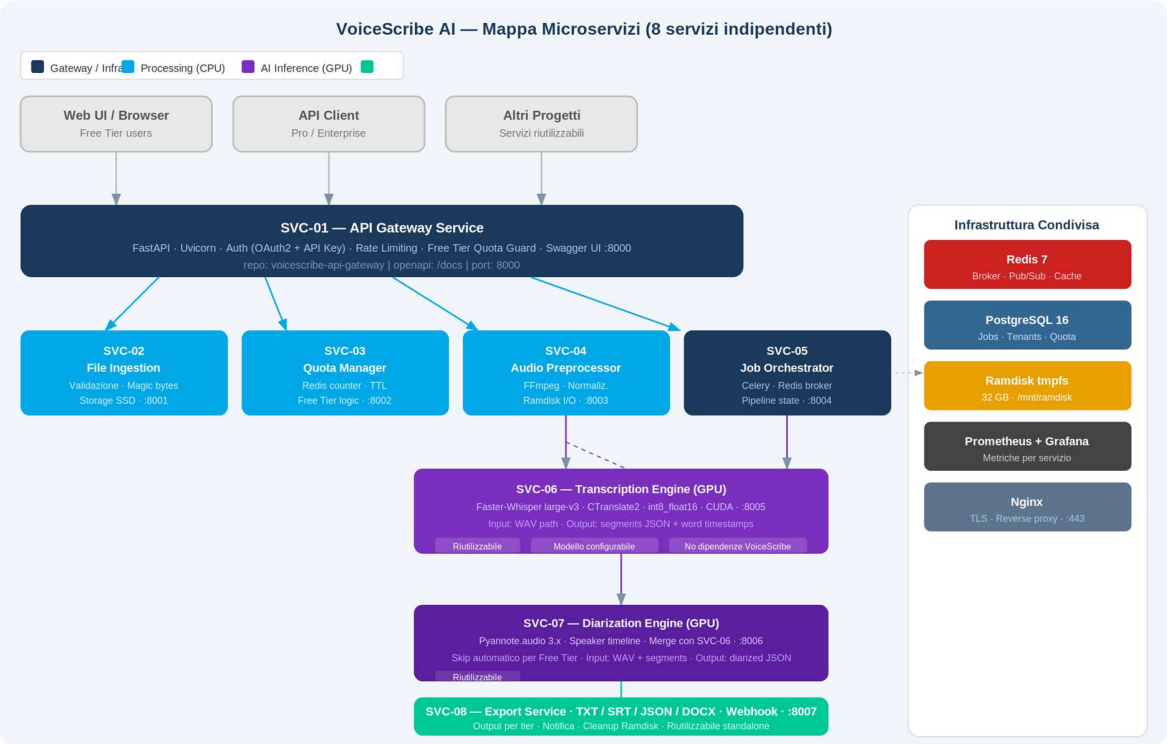

Figura 1 — Mappa completa degli 8 microservizi VoiceScribe AI con porte, responsabilità e infrastruttura condivisa

2.1 Catalogo Microservizi

ID	Nome Servizio	Porta	Hardware	Responsabilita
SVC-01	API Gateway	:8000	CPU	Punto di ingresso unico. Auth (OAuth2 + API Key), rate limiting, routing, Swagger aggregato, Free Tier middleware.
SVC-02	File Ingestion	:8001	CPU	Ricezione multipart upload, validazione formato (whitelist + magic bytes), UUID rename, storage su SSD /data/input.
SVC-03	Quota Manager	:8002	CPU	Gestione quota Free Tier: contatore Redis atomico con TTL, check sincrono, analytics conversione. Riutilizzabile per qualsiasi logica freemium.
SVC-04	Audio Preprocessor	:8003	CPU (multi-core)	FFmpeg: decodifica qualsiasi formato, conversione WAV 16kHz mono, normalizzazione EBU R128, denoising, VAD cut. Output su Ramdisk.
SVC-05	Job Orchestrator	:8004	CPU	Cervello della pipeline: gestisce il ciclo di vita dei job, invia task alle code Celery, aggiorna lo stato in PostgreSQL, gestisce retry ed errori.
SVC-06	Transcription Engine	:8005	GPU (RTX 6000)	Faster-Whisper large-v3 int8_float16 su CUDA. Input: path WAV. Output: segments JSON con word-level timestamps. Completamente standalone.

ID	Nome Servizio	Porta	Hardware	Responsabilita
SVC-07	Diarization Engine	:8006	GPU (RTX 6000)	Pyannote.audio 3.x. Input: WAV + segmenti Whisper. Output: JSON con speaker timeline. Skip automatico per Free Tier. Standalone.
SVC-08	Export Service	:8007	CPU	Generazione TXT/SRT/JSON/DOCX per tier. Webhook POST. Cleanup Ramdisk. Persistenza output SSD. Pub/sub completamento Redis.

2.2 Principio di Riutilizzabilità

Ogni microservizio è progettato seguendo tre regole ferme che garantiscono l'indipendenza completa dal progetto VoiceScribe AI:

- Nessun import cross-service: i servizi non si importano a vicenda come librerie Python. Comunicano esclusivamente via HTTP REST o code Celery.
- Configurazione totalmente esternalizzata: ogni valore variabile — URL Redis, path Ramdisk, modello Whisper, soglie quota — vive nel file .env. Il codice non contiene nessun riferimento hardcoded a VoiceScribe.
- Contratto API dichiarato: l'openapi.yaml è la fonte di verità del servizio. Qualsiasi progetto che voglia integrarlo può leggere il contratto e chiamare gli endpoint senza conoscere i dettagli interni.

Servizio	Riutilizzabile in...	Cosa porta con se
SVC-02 File Ingestion	Qualsiasi SaaS che accetta upload di file (documenti, immagini, video)	Validazione magic bytes, UUID rename, storage organizzato per tenant
SVC-03 Quota Manager	Qualsiasi prodotto con modello freemium — N azioni/giorno gratuite	Contatore Redis atomico, TTL auto-calcolato, analytics conversione, API REST generica
SVC-04 Audio Preprocessor	Podcast editing, call recording normalization, broadcast ingest pipeline	FFmpeg multi-format, normalizzazione EBU R128, denoising, Ramdisk output
SVC-05 Job Orchestrator	Qualsiasi pipeline asincrona multi-stage con stato persistente e retry	Gestione stati, code Celery configurabili, retry esponenziale, metriche Prometheus
SVC-06 Transcription Engine	Subtitling automatico, IT Helpdesk chatbot (KBForge), CRM con note vocali, legal recording	Faster-Whisper su CUDA, modello configurabile via .env, API REST pura senza logica business
SVC-07 Diarization Engine	Meeting recorder, legal transcription (chi ha detto cosa), call center QA	Pyannote.audio standalone, hint num_speakers opzionale, output speaker timeline JSON
SVC-08 Export Service	Generazione documenti da dati strutturati, report pipeline, invoice generator	Template multi-formato (TXT/SRT/DOCX/JSON), webhook generico, cleanup automatico

2.3 Comunicazione tra Servizi

VoiceScribe AI utilizza tre pattern di comunicazione distinti in base alla natura dell'operazione. Nessun servizio conosce i dettagli implementativi degli altri — conosce solo il contratto OpenAPI.

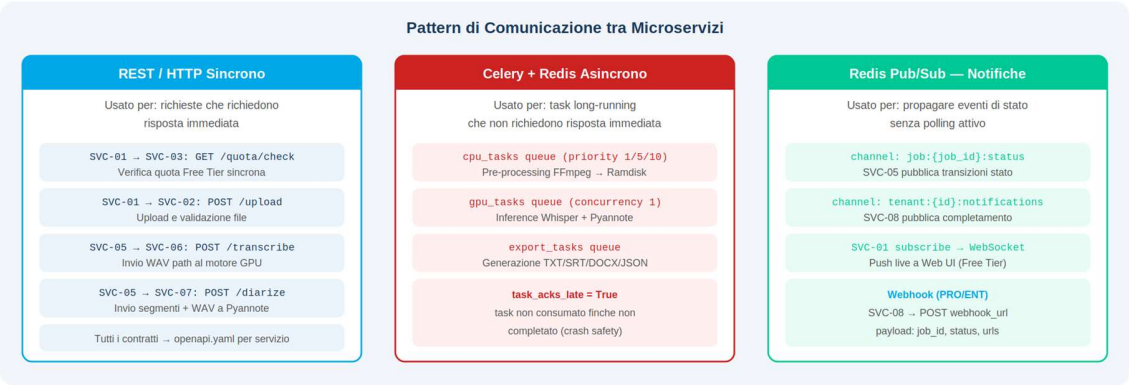

Figura 2 — Pattern di comunicazione: REST sincrono, Celery asincrono e Redis Pub/Sub per notifiche

3. SPECIFICHE OPENAPI / SWAGGER

Ogni microservizio espone la propria specifica OpenAPI 3.1.0. Questo non è solo documentazione: è il contratto vincolante tra i servizi e i loro consumer. FastAPI genera automaticamente lo schema da annotazioni Python (Pydantic models + route decorators), garantendo che spec e codice siano sempre in sync.

Figura 3 — Struttura OpenAPI per microservizio: repo, spec file, endpoint di documentazione e matrice di riutilizzabilità

3.1 Convenzioni OpenAPI Globali

Convenzione	Valore / Descrizione
Versione spec	openapi: 3.1.0 — supporto JSON Schema completo, nullable via type array
Formato versione servizio	MAJOR.MINOR.PATCH — cambiamenti breaking = MAJOR bump

Convenzione	Valore / Descrizione
Endpoint documentazione	GET /docs (Swagger UI) · GET /redoc (ReDoc) · GET /openapi.json (schema raw)
operationId	camelCase univoco globalmente — es: transcribeAudio, checkQuota, uploadFile. Usato da SDK generator.
Tags	1 tag per servizio + tag funzionale. Es: [transcription, audio]. Usato per raggruppare in Swagger UI.
Risposte errore	Schema ErrorResponse standard: {code: str, message: str, detail: any}. Tutti i 4xx/5xx usano questo schema.
Autenticazione	securitySchemes: ApiKeyAuth (header X-API-Key) + BearerAuth (OAuth2 JWT). Definiti una volta in SVC-01, referenziati negli altri.
Formato date	ISO 8601 UTC: 2025-06-15T14:30:00Z. Sempre TIMESTAMPTZ in PostgreSQL.
Paginazione	Query params: limit (default 20, max 100) + offset. Response: {items: [], total: int, limit: int, offset: int}.
x-reusable	Extension custom nei metadata info: x-reusable: true marca i servizi standalone. x-voicescribe-version indica la versione di integrazione.

3.2 Specifiche per Servizio

SVC-01 — API Gateway Service (:8000)

Method	Path	Auth	Descrizione
POST	/v1/transcribe	ApiKey / Session	Upload file audio/video. Trigger pipeline completa. Risposta 202 + job_id. Free Tier: verifica quota e durata prima di accodare.
GET	/v1/jobs/{job_id}	ApiKey / Session	Stato job: status, progress_pct, gpu_inference_ms, download_urls per tier.
GET	/v1/jobs/{job_id}/download/{fmt}	ApiKey / Session	Download output. fmt: txt|srt (FREE), +json|docx (PRO/ENT). 403 se formato non nel tier.
GET	/v1/quota	Session (FREE)	Quota rimanente: {used, limit, reset_at}. Solo per Free Tier.
GET	/v1/jobs	ApiKey	Lista job con filtri. PRO/ENT. Paginazione standard.
DELETE	/v1/jobs/{job_id}	ApiKey	Cancella job QUEUED/FAILED. 409 se in esecuzione.
GET	/v1/stats	ApiKey	Statistiche tenant: job_count, avg_inference_ms, total_hours, queue_depth.
GET	/health + /metrics	Nessuna	Health check sistema + Prometheus scraping.

SVC-02 — File Ingestion Service (:8001)

Method	Path	Descrizione
POST	/upload	Ricezione file multipart. Validazione formato + magic bytes. UUID rename. Storage /data/input/{tenant_id}/{job_id}/{uuid}.{ext}. Risposta: {job_id, path, size_bytes, detected_format}.
GET	/files/{job_id}	Metadata file caricato: path, size, format, created_at.
DELETE	/files/{job_id}	Rimozione file input da SSD (chiamato da SVC-08 al cleanup finale).
GET	/probe/{job_id}	FFmpeg probe sincrono: ritorna {duration_seconds, codec, sample_rate, channels}. Usato da SVC-01 per Free Tier duration check.

SVC-03 — Quota Manager Service (:8002)

Method	Path	Descrizione
GET	/quota/{tenant_id}/check	Verifica quota senza consumarla. Risposta: {allowed: bool, used: int, limit: int, reset_at: datetime}. Usato da SVC-01 prima di accodare.
POST	/quota/{tenant_id}/consume	INCR atomico Redis + scrittura free_tier_usage PostgreSQL. Risposta: {new_count, limit, reset_at}. 429 se gia al limite.
GET	/quota/{tenant_id}/status	Stato completo quota: used, limit, reset_at, quota_exceeded_attempts (storico). Per dashboard admin.
GET	/quota/analytics	Tenant con quota_exceeded_attempts > soglia: candidati upgrade Pro. Parametri: threshold, date_from, date_to.

SVC-04 — Audio Preprocessor Service (:8003)

Method	Path	Descrizione
POST	/preprocess	Avvia task Celery di pre-processing asincrono. Body: {input_path, job_id, output_ramdisk_path, options: {denoise, normalize}}. Risposta: {task_id}.
GET	/preprocess/{task_id}/status	Stato task FFmpeg: PENDING | PROCESSING | DONE | FAILED. Include progress_pct se disponibile.
GET	/formats	Lista formati audio/video supportati da FFmpeg in questa istanza: [{extension, mime_type, supported: bool}].

SVC-05 — Job Orchestrator Service (:8004)

Method	Path	Descrizione
POST	/jobs	Crea e avvia un nuovo job nella pipeline. Invia task a cpu_tasks queue. Risposta: {job_id, status: QUEUED}.
GET	/jobs/{job_id}	Stato completo job con history transizioni: [{status, timestamp, duration_ms}].
POST	/jobs/{job_id}/retry	Re-accoda job FAILED. Solo PRO/ENT. Preserva file sul SSD, elimina WAV Ramdisk se presente.
POST	/jobs/{job_id}/cancel	Revoca task Celery (celery.revoke). Solo su job QUEUED o PREPROCESSING.
GET	/queue/stats	Profondita code: {cpu_tasks: int, gpu_tasks: int, export_tasks: int}. Usato da SVC-01 /health.

SVC-06 — Transcription Engine Service (:8005) [GPU — Riutilizzabile Standalone]

Servizio Completamente Autonomo

- Nessuna dipendenza da VoiceScribe AI — funziona standalone in qualsiasi progetto
- Config via .env: WHISPER_MODEL, WHISPER_COMPUTE_TYPE, WHISPER_BEAM_SIZE
- Input: path locale a file WAV. Output: JSON strutturato con segments e word timestamps
- Casi d'uso esterni: subtitling, IT helpdesk, CRM note vocali, legal recording, podcast

Method	Path	Descrizione
POST	/transcribe	Avvia inference Whisper. Body: {audio_path, language?, beam_size?}. Sincrono — attende completamento GPU. Risposta: TranscriptResult.
POST	/transcribe/async	Versione asincrona via Celery. Risposta: {task_id}. Polling su /tasks/{task_id}/status.
GET	/tasks/{task_id}/status	Stato task GPU: {status, progress_pct, result?}.
GET	/models	Lista modelli Whisper disponibili con VRAM richiesta e RTF stimato.
GET	/gpu/status	Stato GPU: {vram_total_gb, vram_used_gb, vram_free_gb, model_loaded, cuda_version}.

```
# Schema risposta POST /transcribe (openapi.yaml — components/schemas)
TranscriptResult:
  type: object
  required: [job_id, language, duration, segments]
  properties:
    job_id:    { type: string, format: uuid }
    language:  { type: string, example: it }
    duration:  { type: number, description: Durata audio in secondi }
    rtf:       { type: number, description: Real-Time Factor effettivo }
    inference_ms: { type: integer }
    segments:
      type: array
      items:
        $ref: '#/components/schemas/Segment'
Segment:
```

```
  properties:
    start:      { type: number }
    end:        { type: number }
    text:       { type: string }
    confidence: { type: number, minimum: 0, maximum: 1 }
    words:      { type: array, items: { $ref: '#/components/schemas/Word' } }
```

SVC-07 — Diarization Engine Service (:8006) [GPU — Riutilizzabile Standalone]

Servizio Completamente Autonomo

- Standalone: riceve WAV + segmenti trascrizione, restituisce JSON con speaker label
- Casi d'uso esterni: meeting recorder, legal transcription, call center QA, interviste
- Saltato automaticamente per Free Tier (header X-Skip-Diarization: true da SVC-05)

Method	Path	Descrizione
POST	/diarize	Avvia Pyannote pipeline. Body: {audio_path, segments: Segment[], num_speakers?}. Risposta: DiarizationResult con speaker timeline.
GET	/tasks/{task_id}/status	Stato task diarizzazione GPU.
GET	/models/status	Stato modello Pyannote: {loaded, hf_token_valid, vram_mb}.

SVC-08 — Export Service (:8007)

Method	Path	Descrizione
POST	/export	Genera tutti i formati per il tier specificato. Body: {job_id, tier, transcript: TranscriptResult, formats: [txt,srt,json,docx]}. Risposta: {output_paths}.
POST	/webhook/notify	POST payload a webhook_url del tenant (PRO/ENT). Body: {job_id, status, download_urls, completed_at}.
DELETE	/cleanup/{job_id}	Rimuove WAV da Ramdisk e aggiorna completed_at. Chiamato da SVC-05 post-export.
GET	/outputs/{job_id}/{format}	File stream del formato richiesto. Verifica autorizzazione tier prima di servire DOCX/JSON.

3.3 Schema Errori Standard (tutti i servizi)

```
# openapi.yaml — components/schemas/ErrorResponse (condiviso tra tutti i servizi)
ErrorResponse:
  type: object
  required: [code, message]
  properties:
    code:    { type: string, example: QUOTA_EXCEEDED }
    message: { type: string, example: Hai esaurito le 2 trascrizioni gratuite di oggi }
```

```
    detail:  { type: object, additionalProperties: true }
    retry_after: { type: integer, description: Secondi alla mezzanotte UTC (solo 429) }
```

HTTP	code	Quando
400	VALIDATION_ERROR	Body malformato, parametri mancanti o formato file non valido
401	UNAUTHORIZED	API key assente o sessione scaduta
403	TIER_FORBIDDEN	Feature non disponibile nel piano corrente (es: DOCX per Free Tier)
404	NOT_FOUND	job_id o risorsa inesistente
409	JOB_IN_PROGRESS	Tentativo di cancellare un job in esecuzione
413	FILE_TOO_LARGE	File supera il limite di 2 GB
422	DURATION_EXCEEDED	File supera 30 minuti per Free Tier. detail include duration_detected_seconds.
429	QUOTA_EXCEEDED	2 trascrizioni giornaliere esaurite. Header Retry-After con secondi alla mezzanotte UTC.
500	INTERNAL_ERROR	Errore non gestito. Sempre loggato con stack trace via structlog.
503	GPU_UNAVAILABLE	SVC-06 o SVC-07 non raggiungibili o CUDA non disponibile.

4. MODELLO DI ACCESSO E FREE TIER

VoiceScribe AI adotta un modello freemium a tre livelli. Il Free Tier è implementato interamente in SVC-03 (Quota Manager), progettato come servizio autonomo riutilizzabile in qualsiasi prodotto con logica freemium.

4.1 Specifiche Free Tier

Parametro	Valore
Trascrizioni gratuite / giorno	**2 per tenant — gestite da SVC-03 Quota Manager**
Durata massima per file	**30 minuti — verifica via SVC-02 /probe sincrono prima di accodare**
Reset quota	Mezzanotte UTC — TTL automatico Redis in SVC-03
Export disponibili	TXT + SRT. DOCX e JSON: 403 TIER_FORBIDDEN via SVC-08.
Diarizzazione speaker	Saltata: SVC-05 invia X-Skip-Diarization: true a SVC-07.
Priorita coda Celery	1 (vs 5 Pro, 10 Enterprise) — job Free non bloccano mai quelli paganti.

4.2 Implementazione Tecnica — SVC-03 Quota Manager

```
# SVC-03: POST /quota/{tenant_id}/consume
# Logica Redis atomica con TTL auto-calcolato
```

```
def consume_quota(tenant_id: str) -> QuotaResult:
    today = date.today().isoformat()  # UTC
    key = f'free_quota:{tenant_id}:{today}'
    ttl_seconds = seconds_until_midnight_utc()

    # INCR atomico — thread-safe per burst simultanei
    new_count = redis.incr(key)
    if new_count == 1:
        redis.expire(key, ttl_seconds)  # primo job: imposta TTL

    if new_count > FREE_TIER_DAILY_LIMIT:  # default: 2
        redis.decr(key)  # rollback
        raise QuotaExceeded(retry_after=ttl_seconds)

    # Scrivi su PostgreSQL per analytics
    upsert_free_tier_usage(tenant_id, today, new_count)
    return QuotaResult(used=new_count, limit=FREE_TIER_DAILY_LIMIT,
                       reset_at=midnight_utc_timestamp())
```

4.3 Tabella Comparativa Piani

Feature	FREE	PRO	ENTERPRISE
Trascrizioni / giorno	2	Illimitate	Illimitate
Durata max file	30 min	4 ore	Personalizzata
Export TXT / SRT	Si	Si	Si
Export DOCX / JSON	No	Si	Si
Diarizzazione speaker	No	Si	Si
API key + Webhook	No	Si	Si
Priorita coda	Bassa (1)	Alta (5)	Massima (10)
Prezzo mensile	**€ 0**	**€ 29**	**Personalizzato**

5. STACK TECNOLOGICO

Componente	Versione	Ruolo nel Sistema
Python	3.11+	Linguaggio unico per tutti gli 8 microservizi. Type hints obbligatori.
FastAPI	0.111+	Framework REST per ogni servizio. Auto-genera openapi.json da Pydantic + annotazioni. Swagger UI e ReDoc built-in.
Pydantic v2	2.7+	Schema validazione e serializzazione. I model Pydantic sono la fonte di verita per gli schema openapi.yaml.
Faster-Whisper	1.0+	Core SVC-06. 4x piu veloce di OpenAI Whisper via CTranslate2. Compute type int8_float16.

Componente	Versione	Ruolo nel Sistema
Pyannote.audio	3.x	Core SVC-07. Speaker diarization SOTA. Richiede HuggingFace token.
Celery	5.x	Task queue in SVC-04, SVC-06, SVC-07, SVC-08. Code separate con priority per tier.
Redis 7	7.x	Broker Celery + contatori quota SVC-03 + pub/sub notifiche + cache tenant.
FFmpeg	6.x	Core SVC-02 (probe) e SVC-04 (processing). -threads 0 per saturare Threadripper.
PostgreSQL 16	16.x	DB condiviso: jobs, tenants, free_tier_usage. SQLAlchemy async + Alembic migrations.
Nginx	1.26+	Reverse proxy TLS 1.3. Routing /v1/* a SVC-01:8000. Swagger UI pubblica su /api/docs.
Prometheus + Grafana	latest	Ogni servizio espone /metrics. Dashboard GPU inference latency, queue depth, quota usage.

6. MODELLO DEI DATI

6.1 Schema PostgreSQL — Tabelle Principali

jobs

Campo	Tipo	Nullable	Descrizione
id	UUID	NO	PK — generato da SVC-01 al momento dell'upload
tenant_id	VARCHAR(64)	NO	FK tenants.id
tier_at_creation	ENUM	NO	FREE|PRO|ENTERPRISE — immutabile
status	ENUM	NO	QUEUED|PREPROCESSING|TRANSCRIBING|DIARIZING|EXPORTING|DONE|FAILED
priority	SMALLINT	NO	1=FREE / 5=PRO / 10=ENT — usato da Celery
duration_seconds	FLOAT	YES	Valorizzato da SVC-02 /probe. Free Tier: <= 1800s enforced.
ramdisk_path	VARCHAR(512)	YES	Path WAV su /mnt/ramdisk — nullificato dopo cleanup
transcription_raw	JSONB	YES	Output SVC-06 — TranscriptResult schema

Campo	Tipo	Nullable	Descrizione
diarization_raw	JSONB	YES	Output SVC-07 — NULL per Free Tier
gpu_inference_ms	INTEGER	YES	Latenza inference GPU — per Prometheus
celery_task_id	VARCHAR(64)	YES	Per revoca e tracciabilita Celery

6.2 Schema JSON — TranscriptResult (SVC-06 output)

```
{ "job_id": "550e8400-...", "language": "it", "duration": 1847.3,
  "rtf": 0.023, "inference_ms": 42480,
  "segments": [
    { "start": 0.0, "end": 4.2, "text": "Buongiorno...",
      "confidence": 0.94,
      "words": [{"word": "Buongiorno", "start": 0.0, "end": 0.8, "probability": 0.97}]
    }
  ]
}
```

7. FLUSSI APPLICATIVI

7.1 Pipeline Principale — Chiamate inter-servizio

#	Stage	Servizi coinvolti	Dettaglio
1	Ingestione	SVC-01 → SVC-02 → SVC-03	SVC-01 riceve upload. Se FREE: chiama SVC-02 /probe per durata, poi SVC-03 /quota/check. Se ok: SVC-03 /quota/consume. SVC-02 /upload salva file. SVC-05 /jobs crea job. 202 Accepted.
2	Pre-processing	SVC-05 → SVC-04	SVC-05 invia task a cpu_tasks queue. SVC-04 worker: FFmpeg conversione WAV 16kHz, normalizzazione EBU R128, output su /mnt/ramdisk. SVC-05 aggiorna status=PREPROCESSING.
3	Trascrizione	SVC-05 → SVC-06	SVC-05 invia task a gpu_tasks queue. SVC-06: POST /transcribe con audio_path. Faster-Whisper large-v3 int8_float16. Risposta: TranscriptResult JSON. SVC-05 salva in jobs.transcription_raw.
4	Diarizzazione	SVC-05 → SVC-07	Solo PRO/ENT: SVC-05 invia task a gpu_tasks queue. SVC-07: POST /diarize con audio_path + segments.

#	Stage	Servizi coinvolti	Dettaglio
			Risposta: DiarizationResult. FREE: skip, speaker=null in output.
5	Export	SVC-05 → SVC-08	SVC-05 invia task a export_tasks queue. SVC-08: POST /export con tier. FREE: genera TXT+SRT. PRO/ENT: +DOCX+JSON. Salva in /data/output/{tenant_id}/{job_id}/.
6	Notifica	SVC-08 → Redis → SVC-01	SVC-08: webhook POST se PRO/ENT. Redis pub/sub su channel job:{job_id}:status. SVC-01 WebSocket push a browser (Free Tier). SVC-08: DELETE /cleanup/{job_id} per Ramdisk.

8. SICUREZZA

Area	Implementazione
Auth SVC-01	OAuth2 JWT (sessione browser Free Tier) + API Key SHA-256 (PRO/ENT). Rate limit: 10 req/min (FREE), 100 req/min (PRO/ENT). IP allowlist opzionale per Enterprise.
Inter-service auth	I servizi interni (SVC-02..08) esposti solo su rete locale (127.0.0.1 o Docker network). Non raggiungibili dall'esterno. Header X-Internal-Token per mutual auth tra servizi.
OpenAPI security	Security scheme dichiarato in ogni openapi.yaml. Endpoint interni marcati con x-internal: true. Swagger UI di SVC-02..08 non esposta pubblicamente (solo localhost).
Quota enforcement	INCR Redis atomico in SVC-03 (race-condition safe). Guardia secondaria in SVC-04 worker. Nessun bypass via chiamate simultanee.
File upload	SVC-02: whitelist estensioni + verifica magic bytes. UUID rename. Limite 2 GB. Free Tier: rifiuto file >30 min prima del salvataggio (SVC-02 /probe).
Secrets	.env per ogni servizio — mai nel codice. HuggingFace token, Redis password, DB password. Supporto futuro HashiCorp Vault.

9. OTTIMIZZAZIONI HARDWARE

9.1 GPU — RTX 6000 (SVC-06 e SVC-07)

Parametro	Valore Ottimale
compute_type	int8_float16 — VRAM -30%, throughput +20-30% vs float16. SVC-06 large-v3: ~6-8 GB. SVC-07 Pyannote: ~2-3 GB. Totale: ~11 GB su 48 GB.

Parametro	Valore Ottimale
RTF atteso	~0.023 — file 30 min → ~42 s. Free Tier (max 30 min): sempre < 50 s garantiti.
Modello in VRAM	Caricato una volta all'avvio di SVC-06 e SVC-07. Nessun reload tra job. Startup time ~8s incluso nella latenza primo job.

9.2 CPU — Threadripper (SVC-04)

- 12 worker Celery prefork — formula: min(CPU_cores / 2, 16)
- FFmpeg -threads 0 (auto) — ~2 thread per worker = 24 thread FFmpeg simultanei
- Priority queue: job Free Tier (priority=1) non bloccano mai job Pro/Ent (priority 5-10)

9.3 Ramdisk (SVC-04 output / SVC-06-07 input)

```
tmpfs  /mnt/ramdisk  tmpfs  defaults,size=32G,mode=1777  0  0
```

Ramdisk e Free Tier

- File FREE max 30 min = ~29 MB WAV mono 16kHz — occupazione minima
- 12 worker CPU in parallelo = max ~350 MB Ramdisk occupato da job Free
- I 32 GB sono dimensionati per scenari Enterprise con file molto piu grandi
- Monitoraggio: df -h /mnt/ramdisk — alert Prometheus se > 80%

10. DEPLOYMENT & STRUTTURA PROGETTO

10.1 Repository Structure — Monorepo vs Polyrepo

I microservizi sono strutturati come polyrepo: ogni servizio ha il proprio repository Git indipendente. Questo consente di versionarli, deployarli e riutilizzarli in altri progetti senza portare con se codice non necessario. Un repository orchestratore (voicescribe-infra) contiene docker-compose.yml e la configurazione Nginx condivisa.

Repository	Contenuto
voicescribe-infra	docker-compose.yml (Redis, PostgreSQL) · Nginx config · Prometheus · .env.example globale
voicescribe-api-gateway	SVC-01 — FastAPI, auth, routing, Swagger UI pubblica, Free Tier middleware
voicescribe-file-ingestion	SVC-02 — Upload, validazione, FFmpeg probe, storage SSD
voicescribe-quota-manager	SVC-03 — Redis quota, analytics freemium. Riutilizzabile standalone.

Repository	Contenuto
voicescribe-audio-preprocessor	SVC-04 — FFmpeg processing, Ramdisk I/O, Celery CPU workers
voicescribe-job-orchestrator	SVC-05 — Pipeline state machine, Celery routing, retry, metriche
voicescribe-transcription-engine	SVC-06 — Faster-Whisper GPU. Repository completamente indipendente.
voicescribe-diarization-engine	SVC-07 — Pyannote.audio GPU. Repository completamente indipendente.
voicescribe-export-service	SVC-08 — Generatori multi-formato, webhook, cleanup

10.2 Struttura Tipo di Ogni Repository

```
voicescribe-{service-name}/
├── openapi.yaml          # Contratto API — fonte di verita
├── app/
│   ├── main.py           # FastAPI app + openapi metadata
│   ├── routers/          # Endpoint FastAPI
│   ├── schemas/          # Pydantic models → auto-generano openapi.yaml
│   ├── services/         # Business logic
│   └── workers/          # Task Celery (se presente)
├── tests/
│   ├── unit/
│   └── integration/
├── Dockerfile            # Self-contained, no dipendenze esterne
├── requirements.txt      # Solo dipendenze del servizio
├── .env.example          # Tutte le variabili configurabili
├── alembic/              # Migrations (solo servizi con accesso DB)
└── README.md             # Come usare questo servizio standalone
```

10.3 Script di Avvio

```
# SVC-01: API Gateway
uvicorn app.main:app --host 127.0.0.1 --port 8000 --workers 4

# SVC-04: CPU Workers (12 processi)
celery -A app.workers.celery_app worker --queues cpu_tasks --concurrency 12 --pool prefork

# SVC-06 + SVC-07: GPU Workers (1 processo ciascuno)
CUDA_VISIBLE_DEVICES=0 celery -A app.workers.celery_app worker --queues gpu_tasks --concurrency
1 --pool solo
```

11. TIMELINE DI IMPLEMENTAZIONE

Fase	Servizio	Nome	Durata	Deliverable
FASE_00	infra	Setup Infrastruttura	3 gg	Docker Compose · Ramdisk · PostgreSQL schema · Redis
FASE_01	SVC-03	Quota Manager	3 gg	Redis quota + openapi.yaml + analytics PostgreSQL
FASE_02	SVC-02	File Ingestion	3 gg	Upload, magic bytes, probe FFmpeg, openapi.yaml
FASE_03	SVC-01	API Gateway	4 gg	Auth, routing, Free Tier middleware, Swagger UI pubblica
FASE_04	SVC-04 + SVC-05	Preprocessor + Orchestrator	5 gg	FFmpeg pipeline, Celery priority queue, state machine
FASE_05	**SVC-06**	Transcription Engine	5 gg	Faster-Whisper CUDA, openapi.yaml standalone, benchmark RTF
FASE_06	**SVC-07**	Diarization Engine	4 gg	Pyannote.audio, merge con SVC-06, skip Free Tier
FASE_07	SVC-08	Export Service	3 gg	Generatori per tier, webhook, cleanup Ramdisk
FASE_08	ALL	Integration & Go-Live	4 gg	Test end-to-end, load test Free Tier, Prometheus, Nginx TLS

Totale: 34 giorni lavorativi. FASE_05 e FASE_06 (GPU services) possono procedere in parallelo su branch separati dopo FASE_04. SVC-06 e SVC-07 possono essere testati standalone prima dell'integrazione con SVC-05.

12. RISK REGISTER

Rischio	Prob.	Impatto	Mitigazione
Drift tra openapi.yaml e codice	MEDIA	ALTO	FastAPI genera openapi.json automaticamente da Pydantic. CI/CD: test che confrontano /openapi.json con openapi.yaml committato. Fallisce la build se divergono.
Bypass quota Free Tier (burst simultaneo)	MEDIA	MEDIO	INCR Redis atomico in SVC-03 + guardia secondaria in SVC-04. I due controlli in serie rendono il bypass statisticamente impossibile.
Servizio riutilizzato senza aggiornare openapi.yaml	ALTA	BASSO	Versioning MAJOR.MINOR.PATCH obbligatorio. CHANGELOG.md in ogni repository. Breaking change = MAJOR bump + deprecation notice.
CUDA OOM su file Enterprise > 2h	MEDIA	ALTO	SVC-06: auto-split file >90 min in chunk da 30 min con overlap 10s. Non impatta Free Tier (max 30 min).
Worker GPU crash con job in esecuzione	BASSA	ALTO	task_acks_late=True in SVC-06/07. Re-queue automatico. Contatore Free Tier NON incrementato se job fallisce (rollback SVC-03).

Rischio	Prob.	Impatto	Mitigazione
Comunicazione inter-service fragile	BASSA	ALTO	Circuit breaker (tenacity) su ogni chiamata HTTP inter-service. Timeout configurabile per servizio. /health di SVC-01 aggrega stato di tutti i servizi downstream.